GIK

Alg's based on LCP-intervals

→ if $[l,r]$ is lcp-interval of lcp-value h s.t.
$w = S\$[SUF[l]..SUF[l]+h-1]$ is lcp
of $S_{SUF[l]}, \dots, S_{SUF[r]}$, then $[l,r]$ is w-interval

A linear-time alg for finding supermaximal repeats

Supermaximal repeat:

a maximal repeat that never occurs as a substring of any other maximal repeat

eg: taaaaga, maximal repeat $l \geq 2$

aaa aaaa
((1,3),(2,4))
((1,2),(3,4))
a/a aa

↳ only ((1,3),(2,4)) is a supermaximal repeat

Local maximum:

in lcp-table is an lcp-interval $[l,r]$ of lcp-value h satisfying $LCP[q]=h$, $\forall_{q \in [l+1,r]}$

↳ local maximum ~~in ST(S$)~~ in lcp-table w
to a branching node $\overline{w}$ in ST(S$) s.t. ∀edges outgoing from $\overline{w}$ are leaf edges.

↳ local maximum does not contain any other lcp-interval

GLK

A linear time alg for finding supermaximal repeats

Lemma:

A string w of length h is a supermaximal repeat in sequence S, iff w is represented by a local maximum $[l,r]$ in the lcp-table for $S\$$ and chars $S[SUF[l]-1], \dots, S[SUF[r]-1]$ are pairwise distinct.

→ each local maximum covers at least two suffixes

↳ # of local maxima $\leq \frac{n}{2}$, $|S| = n$

→ maximal repeats instead are $O(n^2)$

Alg:

In: SUF & LCP for S
Out: all supermaximal repeats

1) Enumerate all local maxima in the lcp-table of S // $O(n)$
2) $\forall$ local maxima $[l,r]$ check whether the characters $S[SUF[l]-1], \dots, S[SUF[r]-1]$ are pairwise distinct. // $O(\sigma)$
3) If so, report the string $S[SUF[l] \dots SUF[l]+LCP[r]-1]$ as supermaximal repeat

↳ Runtime $O(n\sigma)$, $\sigma = |A|$

GIK

Computation of maximal unique matches

MUMs: maximal unique matches between S and S'.

→ we have $S\#S'\$$

→ $n = |S\#S'\$|$

→ MUM: a maximal unique match is a sequence that occurs exactly once in S and once in S', and is not extendable in any longer such sequence

→ could require minimum length

Lemma:

$T = S\#S'\$$. The string u is a maximal unique match of S and S', iff u is a supermaximal repeat in T represented by an lcp-interval $[\ell, \ell+1]$ of lcp-value $|u|$ s.t.

1) $SUF[\ell] = i$ and $SUF[\ell+1] = j$
2) $u = T[i..i+|u|-1] = T[j..j+|u|-1]$
3) $i < p < j$, $p = |S|$ is position of $\#$ in T

Alg:

In: S, S' and enhanced suffix array for $T = S\#S'\$$
Out: all maximal unique matches

$\mathcal{O}(n)$ 1) Enumerate all local maxima in the lcp-table of $T = S\#S'\$$

$\mathcal{O}(1)$ 2) $\forall$ local maximum $[\ell, r]$ of lcp-value h check ~~whether~~:

I) $\ell + 1 = r$
II) $SUF[\ell] = 0$ or $SUF[r] = 0$ or $S[SUF[\ell]-1] \neq S[SUF[r]-1]$
III) $\min\{SUF[\ell], SUF[r]\} < p < \max\{SUF[\ell], SUF[r]\}$, $p = |S|$

3) If so, report $T[SUF[\ell]..SUF[\ell]+h-1]$ as MUM of length h (CP[r] many) at $\min\{SUF[\ell], SUF[r]\}$ in S and at $\max\{SUF[\ell], SUF[r]\} - (p+1)$ in S'

→ Runtime $\mathcal{O}(n)$

GUK

Exact Pattern Matching in a Suffix Array

→ Pattern $P \in A^m$, determine the positions $j \in [0, n-m]$ s.t. $P = S[j..j+m-1]$

→ Solved by: find all suffixes of S$ which have P as prefix

→ If $[l, r]$ is the m-interval for P, then P occurs in S at positions $SUF[q]$, $\forall q \in [l, r]$

→ decide if m-interval $[l, r]$ for P exists

→ Given m-interval $[l, r]$ for P, the occurences of P in S can be enumerated in $O(r-l)$ time

→ apply binary search to find m-interval

→ l is the first index in SUF, s.t. P is a prefix of $S_{SUF[l]}$

→ r is the last index in SUF, s.t. P is prefix of $S_{SUF[r]}$

→ Method to find m-interval:

process P character by character and perform two binary searches per character in P in the previously found interval.

Exact Pattern Matching on suffix arrays

Alg.:

In: $P \in A^m$, Suffix array SUF for seq $S \in A^n$
Output: P substring of S $\rightarrow$ m-interval for P

```
lcpitv_first (h, l, r, c)   // h-it [l,r] for P[0..h-1], c = P[h]
    q ← ⊥
    while l ≤ r do
        mid = l + (r-l+1)/2
        a = S[SUF[mid] + h]
        if a < c then l ← mid+1          // continue right halve
        else if a = c then q ← mid       // found suffix matching c
        end if                           // but may not be lexicogr. smallest
        r = mid - 1                      // continue left halve
    end while
    return q                             // q ≠ ⊥ => S[SUF[q]] is smallest suffix with c at pos h
end
```

$O(\log(r+l+1))$

```
lcpitv_last (h, l, r, c)    // h-it [l,r] for P[0..h-1], c = P[h]
    q ← ⊥
    while l ≤ r do
        mid = l + (r-l+1)/2
        a = S[SUF[mid] + h]
        if c < a then
            r ← mid - 1                  // continue in left halve
        else
            if c = a then q ← mid
            end if
            l = mid + 1                  // continue right halve
        end if
    end while
    return q                             // q ≠ ⊥ => S[SUF[q]] is largest suffix with c at pos h
end
```

$O(\log(r-l+1))$

Q1K

Exact Pattern Matching on Suffix Array

Alg:

```
lcpitv binsearch (q, l, r, c):
    if c < S[SUF[l]+q] or c > S[SUF[r]+q] then
        return ⊥
    edif
    l = lcpitv first (q, l, r, c)
    if l = ⊥ then
        return ⊥
    edif
    r = lcpitv last (q, l+1, r, c)
    if r = ⊥ then
        return ⊥
    edif
    return (r, l)
end

lcpitvpatsearch (P, m):
    (l, r) ← (0, n)
    for q ← 0 upto m-1 do
        itv ← lcpitvbinsearch (q, l, r, P[q])   // match q+1-interval
        if itv = ⊥ then return ⊥                //  q-interval P[0..q-1]
        else                                    //  already matched
            let (l, r) ← itv
        edif
    elfor
    return [l, r]
end
```

Worst case: interval size is $O(n)$ and we have to execute lcpitvbinsearch m times

$\Rightarrow O(m \log_2 n)$

GK

Algos on enhanced suffix array

Application	Method: Bottom-up	Method: top-down
super-maximal repeats	✓	
maximal unique matches	✓	
minimal unique matches	✓	
maximal repeats	✓	
all pairs suffix-prefix matching	✓	
exact string matching		✓
approx. string matching		✓

Implementation details for suffix arrays

SUF entries integers in range 0 to n

↳ stored in $n \log_2 n$ bits

If $n < 2^{32}$, 32n bits or 4n bytes

Lemma:

If $n < 2^{32}$, the enhanced suffix array requires 9n bytes.

But if we only need SUF → 4n bytes.

4 bit

In practice:

- Store most of the values of table lCP in a table lCP1 using n bytes.

↳ $\forall i \in [1,n]$, lCP1[i] = min{255, lCP[i]}

↳ to access them efficiently, store them in extra table llvtab

↳ contains all pairs (i, lCP[i]), s.t. lCP[i] ≥ 255, ordered by i

↳ each entry requires 8 bytes → 32 bit

↳ if lCP1[i] = 255 ⇒ correct value of lCP is found in llvtab

↳ if we scan the values in lCP1 in consecutive order and find a value 255, then we access the correct value in lCP in the next entry of table llvtab

↳ if we access the value in lCP1 in any order and find a value 255 at i, then we perform a binary search in llvtab using i as key

↳ lCP[i] in $O(\log_2 |llvtab|)$ time

GIK - Ü5

5.3)

Konstruktion eines Suffix Arrays:

Idee: die Suffixe zunächst nach dem ersten Zeichen entsprechend der Alphabetordnung zu sortieren.

In: Sequence S of length n and parameter q

1) Initialize array C of length r^q;
   Counts # of times a q-gram occurs in S, $O(1)$
2) exS = S + [$] * (q-1)
3) Scan exS and count via q-gram code how often a q-gram occurs in exS and save this into C;
   $O(n + r^q)$; where the q-grams are indexed by their code
4) SUF[n]
5) $P(c) = \sum_{c' < c} |C(c')|$, $\forall c \in [0, r^q - 1]$

   $O(r^q)$; partial sums
6) For i = 1 up to n: //$O(n)$
   - -> c = code(exS[i..i+q])
   - -> SUF[P(c)] = i // suffix number
   - -> P(c) = P(c) + 1
7) For every group (q-code) sort group in SUF characterwise;
   $O(r^q n)$

Worst Case:

a^n, this is the worst case:

$O(n + r^q + r^q + n + r^q n) = O(r^q n)$

Burrows-Wheeler Transform and the FM-index

→ Burrows-Wheeler transform is a reversible permutation of the input sequence
↳ decode the original input sequence

→ The bwt of a sequence S is closely related to the suffix array of $S\$$.

→ $S_i = S[i..n-1]\$$, $\forall i \in [0, n]$

→ alphabet Λ is ordered

Def

Suffix array of S is a table SUF of length $n+1$, s.t.

$$S_{SUF[i]} < S_{SUF[i+1]}$$

$\forall i \in [0, n-1]$

Def

Burrows-Wheeler transform bwt for S is a string of length $n+1$, s.t.

$$bwt[i] = \begin{cases} S[SUF[i]-1], & \text{if } SUF[i] > 0 \\ \$, & \text{else} \end{cases}$$

→ $bwt[i]$ = character left to suffix $S_{SUF[i]}$ in S, $SUF[i] > 0$

→ $bwt[i] = \$$ iff $SUF[i] = 0$ → longest suffix for which bwt is not defined

→ F is sequence of length $n+1$, s.t. $F[i] = S[SUF[i]]$, i.e.
F stores the first character of each suffix in sorted order

→ F contains the characters of S alphabetically ordered

GAK

BWT

eg:

	0	1	2	3	4	5	6	7	8	9	10	11
S =	a	g	a	a	t	a	t	a	g	t	a	g

	a	g	t
#occ	6	3	3
cum	0	6	9

i	SUF[i]	suffix	F	bwt
0	2	aatatagtag$	a	g
1	0	agaatatagtag$	a	t
2	7	agtag$	a	t
3	10	ag$	a	t
4	5	atagtag$	a	t
5	3	atatagtag$	a	a
6	1	gaatatagtag$	g	a
7	8	gtag$	g	a
8	11	g$	g	a
9	6	tagtag$	t	a
10	9	tag$	t	g
11	4	tatagtag$	t	a
12	12	$	$	g

Theorem:

For any character a, the i-th occurrence of a in bwt corresponds to the i-th occurrence of a in F.

→ $R_a(X, i)$ be the number of occurrences of a in $X[0..i]$ for any char a, any string X, any position $i \in [0, |X|-1]$

→ $R(X, i) = R_{X[i]}(X, i)$

→ $R(X, i) = q$, then $X[i]$ at position i in X is the q-th occurrence of $X[i]$ in X

→ $R_a(X, i)$ = rank of a in $X[0..i]$

→ 1st occurrence = rank 1

→ Rank-to-First: column mapping

$\alpha_F : \{0, .., n\} \setminus \{\text{Layard}\} \to \{0, .., n-1\}$ defined by $\alpha_F(i) = j$ iff $bwt[i] = F[j]$ and $R(bwt, i) = R(F, j)$

GUK

Inverse BWT

→ LF-mapping allows to reconstruct S in reverse order

→ S_n lexicographically largest suffix

→ last character in S is bwt[n]

$$bwt[n] = S[SUF[n]-1] = S[n-1]$$

→ same char as in F[j], $j = LF(n)$

↳ 1st char of second shortest suffix.

$$S[n-k] = bwt[LF^{k-1}(n)]$$

$$S[n-2] = S[SUF[n-1]-1] = bwt[j] = bwt[LF[n]]$$

Algo

In: sequence length n, bwt, LF for S
Out: sequence S

```
i ← n
for j ← n-1 downto 0 do
    S[j] = bwt[i]
    i ← LF(i)
end for
```

Ch 11

Computing the Last-to-First Mapping

→ count: ~~array indexed by~~

count[a] = # of occurrences of chars in S alphabetically smaller than a.

→ if bwt[i] = a then the char in column F corresponds to bwt[i] is the $R_a(bwt, i)$-th occurrence of a in F

↳ F is alphabetically sorted

↳ this occurrence is in position count[a] + $R_a(bwt, i) - 1$

⇒ $LF(i)$ = count[bwt[i]] + $R_{bwt[i]}(bwt, i) - 1$

= count[bwt[i]] + $R(bwt, i) - 1$

→ R and count can be computed in $O(n + |A|)$

Compute R(bwt, i) for $i \in [0, n]$, $i \neq$ longest col. call count[a], $\forall a \in A$:

```
Let occ be a table indexed by char ∈ A, initial all 0
for i ← 0 upto n do
    if i ≠ largest then
        a ← bwt[i]
        occ[a] ← occ[a] + 1
        R(bwt, i) ← occ[a]
    endif
endfor
Let count be initialized and indexed by char ∈ A
partialsum ← 0
forall a ∈ A in alphabetical order do
    count[a] ← partialsum
    partialsum ← partialsum + occ[a]
endfor
```

→ bwt(S#) can be compressed using the move-to-front (MTF) encoding

CVK

Using BWT for Pattern Search

→ bwt can be used as a full-text index for S

→ LF-mapping reconstructs every substring of S in reverse order!

→ So if P is a substring of S each occurrence of P will be created

→ just perform m steps of reconstruction for ∀ char in P = P[0..m-1] from right to left

→ all occurrences are represented by an m-interval in SUF

and indeed this can be computed step by step using a function related to the LF-mapping

Theorem 4:

Let w be a substring of S, m-1 = |w| and [beg, end] be the (m-1)-interval of w in SUF. Then for any character a, the m-interval [beg', end'] for aw:

$$beg' = count[a] + R_a(bwt, beg - 1)$$

$$end' = count[a] + R_a(bwt, end) - 1$$

if beg' > end' then aw is not a substring of S

GUK

Using BWT for Pattern Search

Alg: Backward Search:

Out: set of suffix-array rows prefixed by $P[0..m-1]$.

succ (A, a) = character following a in alphabetical order.

```
function get_rows(P[0..m-1]):
    a ← P[m-1]
    beg ← count[a]
    lastchar ← last char in A
    end ← if a=lastchar then n else count[succ(A,a)]-1 endif
    i ← m-2
    while (beg ≤ end) and (i ≥ 0) do
        a ← P[i]
        beg ← count[a] + Ra(bwt, beg-1)
        end ← count[a] + Ra(bwt, end) - 1
        i ← i-1
    endwhile
    return if beg > end then ⊥ else [beg, end]
end
```

→ does not use SUF.

→ Alg relies on count of size $|A|$ and the computation of the rank-function $R_a(bwt, -)$

→ FM-Index is loosely defined = a datastructure taking $O(|S|)$ bits of space and supporting rank-operations in $O(1)$

↳ usually constant alphabet size & space have a factor of alphabet size

↳ using get-rows an FM-index can report # of occurrences of pattern P in S in $O(|P|)$ time using $O(n)$ bits space, $n = |S|$

→ Using suffix array in $O(|P| \log n)$ or $O(|P|)$ time using additional tables

↳ using binary search procedure → faster in practice

↳ But uses $O(n \log n)$ space, more than for FM-index

GIK

Using BWT for pattern searches

→ finding positions of P in S

→ Therefore speed of suffix array is much better:

once [beg, end] found then positions are SUF[beg..end]

→ Using FM-index (need more information):

→ store along with the FM-index a sampling of the suffix array

eg: store the values SUF[i] which are multiples of a given value d

↳ then using a hash table or bit array, we mark the rows containing the stored values.

→ To determine the position of occurrences of the pattern 'tag' we first use function get_rows to find the suffix-interval [9,10] for 'tag'.

↳ then we proceed determining the start position of the two suffix in the interval

↳ we only have the marked SUF values {1, 7, 11}

Idea: use the LF-map to move backward in S until we reach a marked position for which the SUF value is known

Alg: (Compute start positions from sampled suffix array):

Function get_position for the computation of SUF[i] on a FM-index enriched with a sample of the suffix array

```
get_position(i):
    offset ← 0
    while row i is not marked do
        i ← LF(i)
        offset ← offset + 1
    calculate
    return SUF[i] + offset
```

→ a marked position is reached in d-1 steps

↳ cost of locating a single occurrence $O(d)$ times the cost of a rank-op

↳ Since cost of storing the SUF values is $O((n \log n)/d)$ bits, the parameter d offers a simple trade-off between memory and speed

GIV.

Fragment Assembly

Shotgun Sequencing Technique:
sequences parts of the DNA (fragments) as follows:
→ sequencing starts at a random position in one of the two complementary strands of the DNA
→ a # of bases is sequenced in canonical direction, from 5' to 3'. This # considerably varies.
→ for each fragment we know the sequence of bases it consists of, but we do not know the direction of the fragment and the start position of the fragments relative to the start of the corresponding strand

Fragment Assembly Problem:
consists of determining the entire target sequence, by assembling the given fragments in the correct orientation order

Phases (Shotgun Sequencing)

Overlap Phase: overlaps of pairs of fragments: if a suffix of one fragment matches a prefix of another fragment

Layout Phase: determine MSA, derived from the positioning of different fragments above each other

Consensus Phase: determine target sequence. choose the base which is dominating a column (majority choice)

Subproblems:
base calling errors,
chimeric fragments,
contamination,
redundancy, fragment orientation
insufficient coverage,
repeats in target sequence

GATK

Base Calling Error

-> incorrectly determined bases in the fragments due to base calling errors
-> fragments contain deletion, insertion or/and replacements of single bases
-> By majority voting and the introduced gap symbol '-', the consensus is not effected by the errors.
-> Error Rate ~ Sequencing Platform
-> Errors are not equally distributed over the sequence positions

Chimeras Fragments

-> Chimera occur if two fragments from different parts of the target sequence are connected to form a pseudo fragment, which does not occur in target seq -> must be identified and ignored before assembly process

eg:
```
--ACCGT--
---CGTGC
TTAC-----
-TACGGT--
--------
TTACCGTGC   instead of TTA---TGC = TTATGC
```

Contamination

-> Sanger Method: fragment sequencing process requires that the DNA is inserted into a cloning vector and cloned in an host organism
-> consequence, sequencing reads often contain segments of sequences from the cloning vector. -> Vector contaminations
-> have to be eliminated from the fragments
  -> compare against vector databases
  -> compare against host organism genome
  -> cross-kingdom comparisons, ie. compare all pairs of seq's from different kingdoms

QUIZ

Unknown Orientation

→ from which strand does the fragment come?

→ f is approximate substring of one strand, iff $\bar{f}$ (reverse complement) is an approximate substring of the complementary strand

→ Each fragment is an approximate substring of the consensus to be determined, either in forward direction or as a reverse complement.

→ There are 2^k combinations

↳ in practice: if two fragments have a significant overlap, then we may assume that they are from the same strand

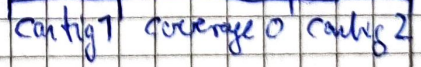
Insufficient coverage

→ coverage at position i of the target sequence is the number of fragments in which there is a base at position i

→ From the total length of the reads and an estimate of the length of the target sequence we can calculate an average coverage per position of the target seq:

$$\frac{\sum_i |r_i|}{\text{target seq length}}$$

→ positions with coverage 0



→ increase coverage → more sequencing

GTK

Repeats in the Target Seq

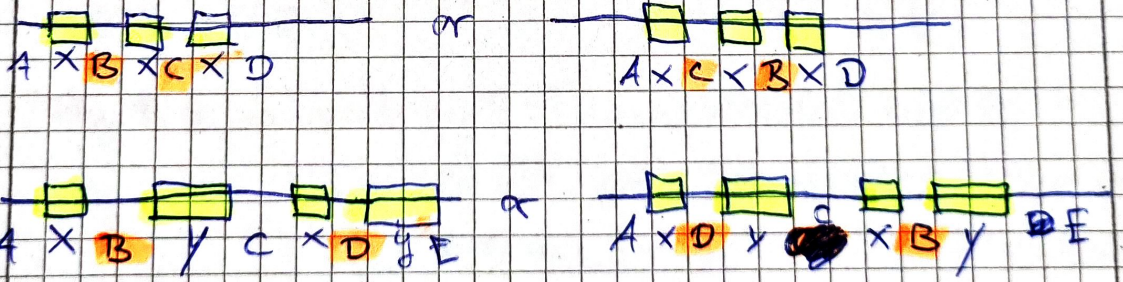
→ duplicated sequence parts

→ Problem if they are larger than the average fragment length

1) If we take into account base calling errors in the fragments, then approximate repeats may falsely be considered as copies of the same part of the sequence. Thus they are placed at the same position ! As a consequence the resulting layout is overcompressed

2) If a fragment is an approximate substring of a repeat, then there are serveral positions in the layout where it can be placed. If we place it at the "wrong" position, then the consensus is weakened.

3) Repeats may lead to an ambiguous fragment assembly problem, one that has more than one possible solution.



→ Solution : use longer reads

GIK

Shortest Common Superstring Model

→ fragment assembly ≈ shortest common superstring problem

↳ used for heuristic algorithm for its solution

→ can be transferred to the other biological more relevant models

Def.:

Let $\mathcal{F}$ be a set of sequences, the fragments to be assembled

→ An $\mathcal{F}$-superstring is a sequence which contains all $f \in \mathcal{F}$ as substrings

→ A shortest $\mathcal{F}$-superstring is an $\mathcal{F}$-superstring of minimum length

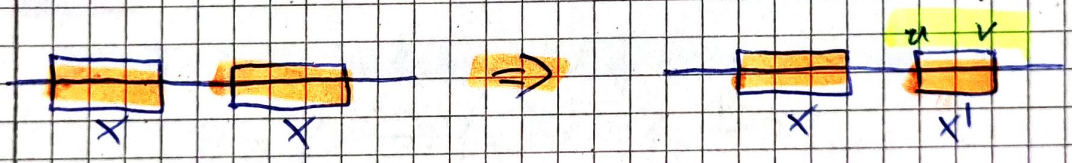
→ SCS-Problem consists of finding a shortest $\mathcal{F}$-superstring

↳ NP-Hard!

↳ ignores errors in fragments: ∀ fragments have to occur as exact substrings in superstring

↳ all are forward direction

↳ But also with these assumptions: Not always correct result ☹



→ length is minimized ⇒ all fragments which are completely in the right copy of X are absorbed by the fragments in the left copy, since in this they also occur as substring

→ also fragments, of which a prefix or suffix overlaps with the sequences in the context of X

↳ coverage inverse: $cov(X) > cov(X')$ $\quad e[X']$ and $r[X']$

↳ connection among the fragments is weak: no fragment that covers X' to the ~~not~~

↳ consensus is rare denotation of two pairs u and v:

1) u begins with the consensus and ends in the middle of X'

2) v begins in the middle of X' and ends with the consensus

MIK Representation of O

Representation of Overlaps

-> long overlap ~ fragments came from same region

-> Assume all forward direction

-> $\mathcal{F}$ - superstring corresponds to paths in certain graphs

Overlap-Multigraph:

$OM(\mathcal{F})$ is a directed graph with node set $\mathcal{F}$ and edge set E_M, s.t. E_M contains an edge $f \xrightarrow{\epsilon} f'$, iff

- $f, f' \in \mathcal{F}$
- $f \neq f'$
- $f = uv$, $f' = vw$, $u,v,w \in \Sigma^*$
- $\epsilon = |v|$

$f \xrightarrow{\epsilon} f' \Rightarrow |f| \geq \epsilon$ and $|f'| \geq \epsilon$, also $\epsilon = 0$ possible

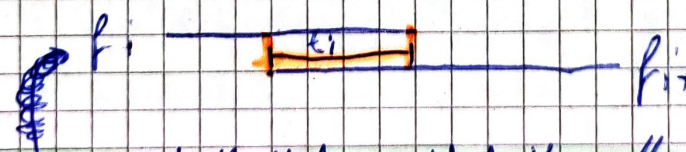
-> $OM(\mathcal{F})$ is a multigraph, since between the same fragments f and f' there may be several edges with different weights

Def:

Let $\mathcal{F}$ be an arbitrary set of fragments and

$$P = f_1 \xrightarrow{\epsilon_1} f_2 \xrightarrow{\epsilon_2} \dots f_n \xrightarrow{\epsilon_n} f_{n+1}$$

be a path in $OM(\mathcal{F})$, s.t. $f_i \neq f_j$ for $i \neq j$. The set of fragments occuring in P is denoted by $\mathcal{F}(P)$



↳ each column of the MSA represented by the path in $OM(\mathcal{F})$ contains the same char

↳ Consensus of MSA is an $\mathcal{F}(P)$-superstring

↳ Consensus of $\mathcal{F}(P)$ is denoted by $S(P)$, is represented by P

↳ $\|\mathcal{F}(P)\| = \sum_{i=1}^{n+1} |f_i|$, $w(P) = \sum_{i=1}^{n} \epsilon_i$

↳ weight of P

GIK

Representation of Overlaps

$$|S(P)| = \|\mathcal{F}(P)\| - w(P)$$

→ each path P in $GH(F)$ represents a $\mathcal{F}(P)$-superstring

→ P is Hamiltonian path if $\mathcal{F} = \mathcal{F}(P)$

↳ all fragments occur exactly once in P

→ A Hamiltonian path P represents an $\mathcal{F}$-superstring

$|S(P)| = \|F\| - w(P)$

→ Maximizing the weight of P ($\Leftrightarrow$ minimizing the length of the Hamiltonian Path $\mathcal{F}$-superstring $S(P)$

→ each Hamiltonian path → $\mathcal{F}$-superstring

→ $\mathcal{F}$-superstring $\not\Rightarrow$ Hamiltonian path

→ $\mathcal{F}$-superstring of minimum length corresponds to Hamiltonian path in $GH(\mathcal{F}')$ where $\mathcal{F}'$ is $\mathcal{F}$ after removal of contained reads

Def

$\mathcal{F}$ is containment-free, if $\forall f, f' \in \mathcal{F}$, $f \neq f'$, we have: f is not a substring of f' and f' is not a substring of f.

Lemma:

Let $\mathcal{F} \subseteq A^*$ be arbitrary. If s is a $\mathcal{F}$-superstring of minimum length, then there is exactly one containment-free set $\mathcal{F}'$ and a Hamiltonian path P in $GH(\mathcal{F}')$ satisfying $s = S(P)$

⇒ create containment-free subset by deleting elements.

GUK

Greedy Algorithm

-> SCS-Problem for Fragment set $\mathcal{F}$ $\Leftrightarrow$ Find Hamiltonian path of max weight in $OM(\mathcal{F})$

-> Hamilton Path:

$P = f_1 \xrightarrow{\ell_1} f_2 \xrightarrow{\ell_2} \dots f_n \xrightarrow{\ell_n} f_{n+1}$ of max weight in $OM(\mathcal{F})$

↳ $\forall\, i \in [1, n]$ and $\forall$ edges $f_i \xrightarrow{\ell} f_{i+1}$ in $OM(\mathcal{F})$ we have $\ell \leq \ell_i$

↳ For each pair of fragments f and f', we can restrict to the edge $f \xrightarrow{\ell} f'$ in $OM(\mathcal{F})$ of max weight ℓ among all edges from f to f'

Def:

Let $OM(\mathcal{F}) = (\mathcal{F}, E_\mu)$. The overlap graph $OG(\mathcal{F})$ is a directed graph with node set $\mathcal{F}$ and edge set E, s.t. E contains an edge $f \xrightarrow{\ell} f'$, iff $f \xrightarrow{\ell} f' \in E_\mu$ and $\forall\, f \xrightarrow{\ell'} f' \in E_\mu$ $\ell' \leq \ell$

Hamilton Path:

1) There is not more than one edge in P leading to the same node
2) There is not more than one edge in P coming from the same node
3) There are no cycles in P

GIK

Greedy Alg

Strategy:

→ Construct a Hamiltonian path edge by edge

→ Consider in each step the edge with maximum weight among all edges which have not been considered before. (Greedy)

→ If by adding this edge to the set of already selected edges, one of the properties 1-3 is violated, reject the edge.

→ As soon as $|F|-1$ edges have been selected, we have constructed a Hamiltonian path

Efficiently checking 1-3:

efficiently implemented as b.f arrays, indexed by fragment numbers

→ table inedges mapping elements from F to boolean values s.t. inedges[f'] = true, iff there is an edge $f \xrightarrow{e} f'$, which has already been selected

→ table outedges mapping elements from F to boolean values s.t. outedges[f] = true, iff there is an edge $f \xrightarrow{e} f'$, which has already been selected

→ A union-find data structure UF with three operations on F:

→ $\forall f \in F$, UF.makeSet(f) creates the set $\{f\}$ in UF

→ $\forall f \in F$, UF.findSet(f) delivers the set in UF containing f

→ $\forall$ disjoint sets $F', F'' \subseteq F$ contained in UF, UF.union(F', F'') creates the union $F' \cup F''$ in UF

implemented by special disjoint-set forests, s.t. for fragment sets F of size n:

A sequence of $m > n$ union/findSet-operations requires $O(m \cdot \alpha(m,n))$ time, where $\alpha(m,n)$ grows so slowly with m & n, that in practical cases we can consider $O(m)$

GIK

Greedy Alg

Alg:

In: $OG(\mathcal{F})$ for a containment-free set of fragments $\mathcal{F}$

Out: edge set representing a Hamiltonian path in $OG(\mathcal{F})$

```
create an empty union-find data structure UF
for all f ∈ F do
    inedges[f] ← false
    outedges[f] ← false
    UF.makeSet(f)
end for
selected ← ∅
edges ← E
while |selected| < |F| - 1 do
    let f → f' be an edge in edges with maximum weight
    edges ← edges \ {f → f'}
    if not inedges[f'] and not outedges[f] and UF.findSet(f) ≠ UF.findSet(f') then
        selected ← selected ∪ {f → f'}
        inedges[f'] ← true
        outedges[f] ← true
        UF.union(UF.findSet(f), UF.findSet(f'))
    end if
end while
return selected
```

set of fragments sets in UF, **invariant properties:**

1) $\forall$ sets in UF are pairwise disjoint
2) each set $\mathcal{F}'$ in UF is a connected component,
   - ↳ there is a path of edges connecting all nodes in $\mathcal{F}'$
   - ↳ there is no selected edge from one fragment $f \in \mathcal{F} \setminus \mathcal{F}'$ to a fragment $f' \in \mathcal{F}'$
   - ↳ there is no selected edge from one fragment $f \in \mathcal{F}'$ to $f' \in \mathcal{F} \setminus \mathcal{F}'$
3) outedges[f] = false ⇒ path in UF.findSet(f) ends with f
4) inedges[f'] = false ⇒ path in UF.findSet(f') begins with f'
5) the new edge and the union operation leads to a set of fragments, s.t. there is a path of selected edges connecting all fragments
6) each step, as the corresponding connected components of f and f' are different, the selected edge f → f' does not lead to a cyclic path of selected edges.

GTK

Greedy Alg

⇒ $n = |\mathcal{F}|$, $O(n^2)$ edges in $OG(\mathcal{F})$

→ length of overlaps range 0 to $\ell - 1$, where $\ell = \max\{|f| \mid f \in \mathcal{F}\}$

→ use counting sort algorithm, we can sort edges in $O(\ell + n^2)$ time

↳ find next max in const time

→ Condition in if-statement are evaluated at most $O(n^2)$ times

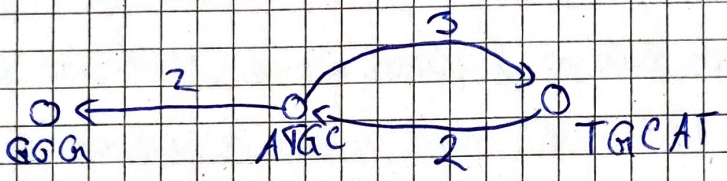
→ eval condition in $O(1)$

→ Union operation is applied $n-1$ times at most $O(n)$ time

⇒ total $O(\ell + n^2)$

→ greedy-alg is heuristic → does not always find a min length $\mathcal{F}$-superstring

eg:



Greedy : ATG.CAT.GCC , length = 9

Optimal : TG.CAT.GCC , length = 8

GIK

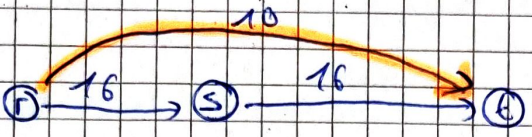
From overlap graphs to string graphs

→ To solve the FAP, we can restrict ourselves to containment free sets $\mathcal{F}$ of fragments.
↳ reduces size of $OG(\mathcal{F})$

→ Optimize: transitive edges

Def:
An edge $r \xrightarrow{\ell''} t$ in $OG(\mathcal{F})$ is transitive, iff there are two edges $r \xrightarrow{\ell} s$ and $s \xrightarrow{\ell'} t$, s.t. $\ell + \ell' = |s| + \ell''$
(the $|s|$ self)

Eg:



, $|s| = 12$

→ transitive overlaps do not deliver any additional information. For solving the FAP, as the overlapping sequence represented by the transitive overlap is a proper substring of two other overlaps
↳ s is neither contained in r nor in t, $\ell \leq |s|$ and $\ell' < |s|$,
↳ $\ell - |s| < 0$ and $\ell' - |s| < 0$
↳ $\ell'' = \ell + \ell' - |s| < \ell$ and $\ell'' = \ell' + \ell - |s| < \ell'$
↳ so $t[1 \ldots \ell'']$ is a proper prefix of $t[1 \ldots \ell']$
$r[|r| - \ell'' + 1 \ldots |r|]$ is a proper suffix of $r[|r| - \ell' + 1 \ldots |r|]$
↳ remove transitive edges

⟷ $OG(\mathcal{F})$ after removal of contained reads and transitive edges is called string graph

From overlap graphs to string graphs

→ Property of string graph:
each genuine assembly is a valid path in the string graph (and vice versa)

→ String graph thus represents all relevant information inferable from overlaps

→ Subgraphs representing repeats are traversed multiple times

→ Strip Graph traversal:
computes suffix-prefix matches using an algorithm based on the Burrows et Wheeler transform, allowing to classify suffix-prefix matches as transitive or non-transitive, so that the string graph can directly be constructed.

GUK

Clustering of gene expression data

Clustering genes by their expression vectors

Goal: cluster the set of all yeast genes based on their expression vectors

Since # clusters not always known. Run clustering with different values of k. → select k s.t. the results makes sense biologically.

Gene Expression Clustering is equivalent to partitioning a collection of n points in m-dimensional space into k clusters

Clustering by fast search and find of density peaks

→ density peak clustering

↳ set P of points in m-dimensional space.

Def:

Euclidean distance $d_e(\alpha, \beta)$, $\alpha, \beta \in P$

$$d_e(\alpha,\beta) = \sqrt{\sum_{i=1}^{m} (\alpha[i] - \beta[i])^2}$$

→ based on a constant $d_c > 0$, the cutoff distance

↳ neighbor of point $\alpha \in P$ has euclidean distance $< d_c$ to α

↳ sort all pairwise distance values and take the p_c percent smallest distances

↳ the next distance is d_c

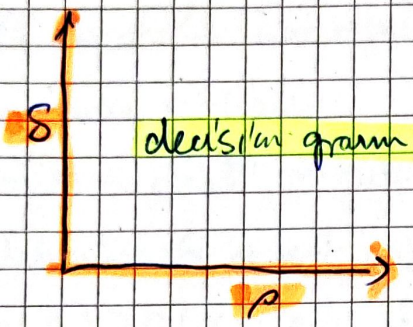
CluK

Clustering by fast search and find of density peaks

local density $\rho(\alpha)$ of point α = # of points $\beta \in P \setminus \{\alpha\}$ satisfying $d_c(\alpha,\beta) < d_c$

distance $\delta(\alpha)$ of a point α:

$$\delta(\alpha) = \begin{cases} \max\{d_c(\alpha,\gamma) \mid \gamma \in P\} & \text{, if } \rho(\gamma) \leq \rho(\alpha),\ \forall \gamma \in P \\ \min\{d_c(\alpha,\gamma) \mid \gamma \in P, \rho(\gamma) > \rho(\alpha)\} & \text{, else} \end{cases}$$



Local density based on gaussian filter

→ apply gaussian filter instead of usual threshold

$$g(d) = \exp\left(-\left(\frac{d}{d_c}\right)^2\right)$$

→ PDF of gaussian distribution with mean μ and standard deviation σ

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

→ local density $\rho_g(\alpha)$:

$$\rho_g(\alpha) = \sum_{\gamma \in P} g(d_c(\alpha,\gamma))$$

↳

$$\delta_g = \begin{cases} \max\{d_c(\alpha,\gamma) \mid \gamma \in P\} & \text{, if } \rho_g(\gamma) \leq \rho_g(\alpha),\ \forall \gamma \in P \\ \min\{d_c(\alpha,\gamma) \mid \gamma \in P, \rho_g(\gamma) > \rho_g(\alpha)\} & \text{, else} \end{cases}$$

Grid

From decision diagram to clusters

→ Points with max density are good points for center of cluster.

Alg: Density peaks clustering

- ⇒ compute ρ and δ from pairwise euclidean distance of the points
- ⇒ points with large δ-value and relatively high ρ-value, according to user-defined threshold, are considered the cluster centers.
- ⇒ the threshold can refer to the # clusters and / or to the minimum ρ-Value.
- ⇒ For each point α s.t. $p(\alpha)$ is not the maximum value, determine the nearest neighbor of α, i.e. the point γ, s.t. $\rho(\gamma) > \rho(\alpha)$ and $\delta(\alpha) = d_e(\alpha, \gamma)$
- ⇒ cluster membership is inherited by the nearest neighbor relation in descending order of p-Values, i.e. if a nearest neighbor γ of α belongs to a cluster, so does α
- ⇒ singleton are those points with low density and large δ-value exceeding some user defined threshold

⇒ each step get $\max p_g(\alpha_i)$ and put α in same cluster as $NN(\alpha_i)$ = nearest neighbor of α

⇒ $p_g(\alpha_i) \leq 0{,}05 \Rightarrow$ Singleton

GNK

Implementation of Density Peaks Clustering

→ class DensityPeaks

→ np.percentile ⇒ // , iterates much faster than a loop in Python

→ n×n matrix B, B(i,j) is True iff dist_mat[i,j] < d_c

Running Time

- n = # points , k = # clusters
- compute d_c from p_c to smallest distances $s = \frac{p_c}{100} n^2$ // $O(n^2 \log s)$
- $s \in O(n^2) \Rightarrow O(n^2 \log(n^2))$
- compute densities of n^2 entries // $O(n^2)$
  - ↳ n rows each with n values ⇒ $O(n^2)$ time
- → sort densities $O(n+q)$, q = # different densities
  - ↳ gaussian kernel apply comparison based sorting: $O(n \log n)$ time
- → computation of nearest neighbor $O(n^2)$

Task	Running Time
Initialization of variables	$O(n)$
compute d_c	$O(n^2 \log n^2)$
compute densities	$O(n^2)$
sort 2 different discrete densities	$O(n+q)$
sort floating point densities	$O(n \log n)$
compute nearest neighbor	$O(n^2)$
compute max-dist	$O(n)$
compute δ	$O(n)$
Assign cluster IDs	$O(n)$
compute cluster sets	$O(n \log n)$
overall	$O(n^2 \log n^2)$

red-black tree ← $O(\log s)$ time per insertion of set size s